

GAMEFIX

Relatório de Integração com MOM – Sprint 2

LDAMD | PUC Minas – Engenharia de Software | 1º Semestre 2026

1. Introdução

Este relatório descreve as decisões de design relativas à integração do GameFix com um Middleware Orientado a Mensagens (MOM) na Sprint 2. O objetivo desta sprint foi evoluir o sistema da comunicação puramente síncrona — estabelecida na Sprint 1 via REST — para uma arquitetura orientada a eventos (EDA), em que os componentes do sistema se comunicam de forma assíncrona por meio de filas de mensagens.

2. Escolha da Ferramenta: RabbitMQ via CloudAMQP

O RabbitMQ foi escolhido como broker de mensagens pelos seguintes motivos:

- Protocolo AMQP 0-9-1: protocolo maduro com garantias de entrega, persistência de mensagens em disco e suporte a acknowledgements (acks), o que elimina o risco de perda de eventos em caso de falha do consumidor.
- Filas duráveis: as filas foram configuradas com `durable: true` e as mensagens com `persistent: true`, garantindo que os eventos sobrevivam a reinícios do broker.
- Prefetch e fair dispatch: o worker usa `prefetch(1)`, processando uma mensagem por vez e garantindo distribuição justa em cenários com múltiplos consumidores (padrão `competing consumers`).
- CloudAMQP: por não haver Docker ou Podman disponível no ambiente de desenvolvimento, optou-se pelo plano gratuito do CloudAMQP (Little Lemur), que oferece conectividade AMQPS (TLS) sem necessidade de infraestrutura local.

3. Padrão Utilizado: Work Queue com Roteamento por Fila Dedicada

O padrão adotado é o Work Queue (também chamado Task Queue), conforme descrito por Hohpe e Woolf (2003) em *Enterprise Integration Patterns*. Cada tipo de evento do GameFix possui uma fila dedicada, o que oferece as seguintes vantagens:

- Isolamento: um evento de alta frequência (ex: `chamado.criado`) não bloqueia o processamento de eventos de outra fila.
- Observabilidade: o painel de gerenciamento do RabbitMQ exibe o backlog de cada fila individualmente.
- Escalabilidade futura: é possível adicionar workers especializados para filas específicas sem alterar o produtor.

Optou-se por NOT usar o padrão fanout/topic exchange nesta sprint pois o sistema ainda tem um único consumidor (o worker). A evolução para exchanges do tipo topic é prevista para a Sprint 4, quando os dois apps Flutter precisarão receber eventos de formas distintas.

4. Documentação dos Eventos

Evento	Fila RabbitMQ	Produtor	Consumidor	Trigger
chamado.criado	gamefix.chamado.criado	Backend	Worker → App Técnico	POST /api/chamados
chamado.aceito	gamefix.chamado.aceito	Backend	Worker → App Cliente	PATCH .../aceitar
chamado.recusado	gamefix.chamado.recusado	Backend	Worker → App Cliente	PATCH .../recusar
orcamento.enviado	gamefix.orcamento.enviado	Backend	Worker → App Cliente	POST .../orcamento
orcamento.aprovado	gamefix.orcamento.aprovado	Backend	Worker → App Técnico	PATCH .../responder
orcamento.recusado	gamefix.orcamento.recusado	Backend	Worker → App Técnico	PATCH .../responder
chamado.concluido	gamefix.chamado.concluido	Backend	Worker → App Cliente	PATCH .../concluir

5. Estrutura do Envelope de Mensagem (Payload JSON)

Todas as mensagens publicadas nas filas seguem o mesmo envelope, garantindo consistência para os consumidores:

```
{
  "event_id": "550e8400-e29b-41d4-a716-446655440000", // UUID único por mensagem
  "event_type": "chamado.criado", // nome do evento
  "timestamp": "2026-05-25T20:00:00.000Z", // ISO 8601
  "payload": { // dados específicos do evento
    "chamado_id": "uuid-do-chamado",
    "cliente_id": "uuid-do-cliente",
    "aparelho": "PlayStation 5",
    "modelo": "CFI-1215A"
  }
}
```

6. Fluxo de Comunicação Assíncrona

O diagrama abaixo descreve o fluxo de um chamado sendo criado, demonstrando que a comunicação entre Backend e Worker ocorre exclusivamente via MOM, sem chamada REST direta:

```
Cliente HTTP      Backend (Express)      RabbitMQ      Worker
|                 |                       |             |
|-- POST /api/chamados -->|               |             |
```

```
|                                     |-- INSERT eventos -->| (auditoria DB) |
|                                     |-- sendToQueue ----->| gamefix.chamado.criado
|<-- 201 Created -----|                                     |
|                                     |-- consume ----->|
|                                     |                                     |-- handler()
|                                     |                                     |-- ack()
```

A resposta HTTP 201 é enviada ao cliente ANTES do worker processar o evento, demonstrando a assincronicidade real do sistema: o backend não espera a conclusão do processamento para responder.

7. Desafios Encontrados e Soluções Adotadas

Desafio	Solução Adotada
Ambiente sem Docker/Podman	Uso do CloudAMQP (plano gratuito Little Lemur) com conexão AMQPS (TLS), eliminando a necessidade de infraestrutura local.
Publicação assíncrona sem bloquear o HTTP	A função publicarEvento() inicia a publicação no RabbitMQ de forma não-bloqueante (sem await na chamada dos controllers). Erros de MOM são logados mas não retornam erro HTTP ao cliente.
Auditoria de eventos da Sprint 1	Mantida a gravação síncrona na tabela 'eventos' do banco, funcionando como outbox simplificado: se o RabbitMQ estiver fora, o evento ainda é registrado e pode ser reenviado manualmente.
Reconexão automática ao RabbitMQ	O eventoModel.js implementa reconexão com backoff de 5 segundos ao detectar fechamento inesperado da conexão AMQP.

8. Referências

HOHPE, G.; WOOLF, B. Enterprise Integration Patterns: designing, building, and deploying messaging solutions. Boston: Addison-Wesley, 2003. (Fundamenta os padrões Work Queue, Competing Consumers e Message Channel adotados no projeto.)

COULOURIS, G. et al. Distributed Systems: Concepts and Design. 5. ed. Harlow: Addison-Wesley, 2011. (Fundamenta o conceito de comunicação indireta e desacoplamento temporal adotado na arquitetura orientada a eventos do GameFix.)

RABBITMQ. Consumer Acknowledgements and Publisher Confirms. Disponível em: <https://www.rabbitmq.com/docs/confirms>. Acesso em: 25 mai. 2026. (Fundamenta o uso de durable queues, persistent messages e acknowledgements implementados no worker.js.)

RICHARDSON, C. Microservices Patterns: with examples in Java. Shelter Island: Manning, 2018. (Fundamenta o padrão Outbox simplificado adotado para garantir auditoria de eventos mesmo em caso de falha do broker.)